

Crinkler: The Real-World Use of an Esoteric Compression Algorithm

Victor Veytsman

1 Introduction

Often-times in the Computer Science world, a brilliant paper will be published, an excellent algorithm made that elegantly solves a problem in a way never seen before, only to never be used, for practical reasons. It might take up too much space, or be too slow, or be too hard to implement, or only work well for an edge case too specific to consider. Whatever the reason, there is often a disconnect between the cutting edge of Computer Science and what people end up using on a day-to-day basis. This article delves into software called *Crinkler*, which demonstrates how algorithms that would otherwise be restricted to a few scientific papers buried in some university's archive can be used in the real world. But before we take a look at Crinkler, we must examine what spawned the software in the first place: the *Demoscene*.

1.1 What is the Demoscene?

The Demoscene is a fascinating subsection of Internet culture that brings people together to push their code to its limits. “Sceners,” the demonym for those who participate in this culture, create “demos” which assemble impressive music and graphics into programs that perform awe-inspiring stunts. There are many different types of demos, most putting a unique constraint on the demo that works to highlight the scener's creativity. The most common category is that of “intros,” which are restricted by their size, which is the amount of disk space they can take up. Other categories include different retro platforms, e.g., computers like the Amiga or the Commodore 64, where their low level of performance provides the restrictions.

The Demoscene has its roots in the early 1980s when computers first started entering the home. Games released for these computers had copy protection, to make sure people didn't spread them illegally, so lots of people, calling themselves “crackers,” worked on breaking this protection to share the game for free. When they cracked a game, they would often add an introduction to it that shouted them out. As the 80s went on, crackers would keep trying to one-up each other, fitting impressive graphics into the little space they had. They also met each other at parties to swap their cracked games.

Nowadays, the Demoscene lives primarily in Europe, having moved away from cracked games to original creations. Demoparties are hosted in countries like Germany, Switzerland, Denmark, Norway, and Sweden, and many Internet forums are dedicated to hosting demos and discussions about them.

2 What is Crinkler? How does it work?

To pull off their impressive stunts, demos employ a variety of tricks to make their code smaller and more efficient. A lot of demos rely on what are called “demotools,” which are tools created to make common

tasks easier and better. One of these very common tasks is compression, which is essential in the creation of intros, so there are many demotools whose primary purposes are compressing the code as efficiently as possible. Many of these tools fall into the category called “packers,” because they compress the demo and then add a small program onto it, called an “unpacker.” When the program is run, the unpacker runs first and decompresses the main demo.

Crinkler is one of these packers, made specifically for 4-kilobyte intros, programs that can only contain 32,000 zeros and ones (an amount of data equivalent to about one page of text). Crinkler is special, though, because it also acts as a *linker*. When you have code among a bunch of different files, the compiler generates a different *object file* for each code file. The object file is mostly machine code, code that the computer can read, but it also has some information for the linker to use. The linker then takes all of these files and puts them together into one file that the computer can run. Because Crinkler is also a linker, it can combine these object files in a way that makes them compress more efficiently, putting similar parts closer to each other and splitting up the code and the data according to their different structures. Once Crinkler finishes the linking process, it begins compression, taking advantage of several different algorithms.

2.1 Arithmetic Coding

Crinkler uses a technique called *arithmetic coding* to compress the file. Arithmetic coding is a compression algorithm that compresses the given data into a single number based on the probability of each symbol in that data occurring after another symbol. Crinkler makes this easier by going down to the bit level—it works with zeros and ones, whereas other implementations may work with bytes (eight zeros and ones) or even bigger blocks. To demonstrate how this algorithm works, here is an example:

Suppose there is a sequence of bits that is “0110.” Let’s say the probability table of this example is 0.6 for a zero and 0.4 for a one—it doesn’t matter whether or not this is perfect for the given sequence, and the creation of these probability tables is discussed later. The first step the algorithm takes is dividing the interval between 0 and 1 into smaller sections based on those probabilities. In Figure 1, the numbers on the left represent the interval between 0 and 1, and on the inside of each box is the sequence that the box’s interval describes. Since our sequence starts with a 0, we split our interval once again on the 0, as can be seen in Figure 2. Now, we have an interval between 0 and 1 that represents the sequence “01”, which is the start of our sequence, so we want to split that interval again. In Figure 3, this process has been completed twice to bring us to the final interval that we need to describe our full sequence, which is between 0.4 and 0.4576.

The algorithm can output any number that is within the interval. However, it would be most advantageous to output the shortest possible binary number that is within that interval. Finding this binary number is left as an exercise to the reader.

Now that we have the encoding done, we need a program to decode back to our original input. This is the beauty of arithmetic coding: the decoder is extremely simple, especially when you’re only dealing with zeros or ones. In essence, it does the same thing that the encoder does, but backwards. Given the encoded number, it divides the range between 0 and 1 into two sections. Then, it determines the section of the encoded number and outputs the bit corresponding to that section. Finally, it subdivides that section into two sections again, repeating the process for the length of the compressed code. When executing programs, the computer reduces them to instructions, where each instruction is then executed by the CPU. The developers of Crinkler have managed to get their decoder program to run with only 18 instructions,

each instruction not more than a few bytes (most being only one byte).

Figure 1: The first division of the interval



Figure 2: The second division of the interval

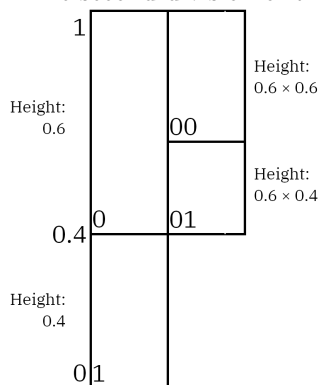
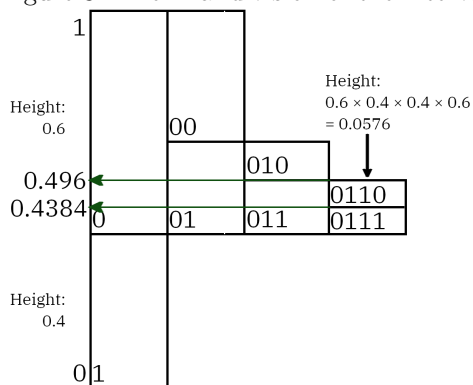


Figure 3: The final division of the interval



2.2 Making Probability Models

2.2.1 Making one Probability Model

With our previous example, a probability table said that there would be a 60% chance a 0 would occur and a 40% chance a 1 would occur in the sequence. These numbers were chosen as an example, but present an interesting question: how does the program make the probability models as accurate as possible? More accurate probability models should generate bigger intervals, which would make it easier to find a short number in that interval.

To create a model, Crinkler reads through the data, keeping track of the probabilities that bits occur and updating them as it reads new data. It stores a model for each context, the context being the previous bits it has read. Each model consists of two counters which count the number of times zeros and ones occur after their contexts. The algorithm slightly changes these counters in order to prioritize recent events over older ones. It does this by seeing if the opposite counter (the one whose bit didn't come after the model's context in the set the algorithm was reading) is greater than two, and if so, it divides it by two and adds one.

For example, let's take the sequence 010101100. Let's say that we have one model for this sequence whose historical data is 1 bit long. This means that the model is calculated twice: once for the historical data "0" and once for the data "1." The algorithm would keep track of all of the counters for each historical possibility at once, but for simplicity's sake, let's calculate just the model for each 0.

1. In the first pair of bits, 01, we add 1 to the counter: ($c_0 = 0, c_1 = 1$)
2. In the second pair of bits, we see the same thing: ($c_0 = 0, c_1 = 2$)
3. In the third pair, again: ($c_0 = 0, c_1 = 3$)
4. In the fourth pair, we see a 00, so we add 1 to c_0 , and then manipulate the counter as previously mentioned, rounding down: ($c_0 = 1, c_1 = \frac{3}{2} + 1 = 2$)

Now that we have calculated the counters for the 0 model, we can find the probability that a 1 occurs with this formula: $p(1) = \frac{c_1}{c_0 + c_1}$. Of course, it is impossible for $p(1)$ to be 0 or 1, so often a small number will be added to each counter to make sure that this impossibility never occurs; this is called a bland model. Models are added by simply adding their counter values, so adding our calculated model and our bland model (in which c_0 is 1 and c_1 is 1) gives us $p(1) = \frac{2+1}{(1+1)+(2+1)} = 0.6$. We estimated a 60% chance that after every 0 there will be a 1.

2.2.2 Making Many Probability Models and Combining Them

This way, Crinkler can calculate the probability of a zero or one occurring. However, it takes this whole procedure one step further by calculating 255 models. Each model is assigned a number from 0 to 255, which is 0b11111111 in binary. (The prefix 0b means that it's a binary number.) These numbers act as a *bit mask*; when converted to binary, the ones indicate which bytes are part of the historical data. This is seen more clearly in Figure 4 where each box represents a byte, with the green box indicating the one currently being read. With the zero model, only that byte is considered in the context; with the 128 model, the most recent byte is considered in the context (the binary number is backwards); with the 129 model, it's the 8th most recent and most recent bytes being part of the historical data; and with the 255 model, all of the most recent 8 bytes are part of the historical data. The reason Crinkler does this is compiled code will often have repeating patterns of the type "every fifth letter is an O." If that occurs often in a section of code, then a model that considers the 5th most recent byte will be very good at compressing that

data. Of course, computing these many models with all of their different possible historical contexts is very expensive, meaning it takes a lot of time and computer memory. But it doesn't matter for Crinkler, because it is working on very small amounts of code.

Figure 4: Probability model

data	A	B	C	D	E	F	G	H	*
0 model	0	0	0	0	0	0	0	0	*
128 model	0	0	0	0	0	0	0	1	*
129 model	1	0	0	0	0	0	0	1	*
153 model	1	0	0	1	1	0	0	1	*
255 model	1	1	1	1	1	1	1	1	*

Combining these models ends up being quite easy. Given a set of counters and weights (for a specific context), the formula for the probability that one occurs after that context is shown in this equation:

$$p(1) = \frac{\sum \frac{w * n_1}{n_0 + n_1}}{\sum w}$$

In this equation, n_1 is the one counter for that context (for each model), n_0 is the zero counter (for each model), and w is that model's weight. There are several different ways to determine the weight associated with each model. The algorithm could calculate a weight as the algorithm goes along, which involves neural networks and other interesting computer science features, but Crinkler does not do this because experiments have shown that the decompressor (the part of the code that is attached to the final executable) is made around 100 bytes longer using this method, and its gains in compressibility do not offset that loss. Instead, Crinkler makes rough estimates by setting the weight to the number of bits that are on in the bit mask (the number of ones in a model in Figure 4). Crinkler gives the user an option to use slow compression, which then optimizes these weights a little further.

2.2.3 Choosing Which Models to Combine

Now, Crinkler needs to determine which of these models is the best for each section of code. This procedure—choosing which models fit which sections—is largely why Crinkler acts as a linker; as previously stated, different types of data are split up so that similar data can be grouped together, and models for these data can be stronger. To determine which models are the best to use, the algorithm could, as previously stated, calculate the weights as it goes along and then select the models with the best weights. However, because it doesn't do that, it solves it in a way that is much simpler, if not as ideal. It loops through the models, and for each model, checks whether or not compression improves. After that, it loops through the existing models in the set and checks whether removing any model improves compression. This might be a slower or clunkier solution, but because Crinkler is working with such small files, it doesn't really make a big difference.

Unfortunately, working with so many contexts and models and counters, while great in the theoretical world, is extremely difficult with real data, on real computers. It means that when compressing and decompressing, all of these counters take up a lot of computer memory, which can make programs extremely

slow or completely unusable. To get around this, Crinkler pulls off one last little trick: it uses a hash table. A hash table basically just associates a set of values with a set of keys, so if you have a key you can look into the table and find a value. This improves decompression because Crinkler can turn a distinct historical context into a key by “hashing” it, turning the context into a key, and looking it up in the table, where it stores the context’s probabilities in the form $\frac{n_0}{n_1}$, referencing the two counters from earlier. It can then define the size of the table and keep it from exploding computer memory. Of course, this begs the question: how can a hash table store the same data in a more limited space? The answer is that it doesn’t. Certain contexts can map to the same key even if they are very different, and the chance that this happens goes up as the size of the table goes down. However, Crinkler doesn’t do anything about this problem, choosing to ignore it to keep the size of the decompressing code small. This can throw off the probability calculations for models, making the interval at the end of the arithmetic coding algorithm unnecessarily small, but it doesn’t increase the size by as much as if code to handle collisions were put into the decompressor.

3 Conclusion

Thus, Crinkler compresses intros and demos drastically. It acts as a linker which groups similar data together and optimizes code that Windows, the operating system, needs to run the program; it uses an arithmetic coding algorithm to compress the data; it supplies that coder with models that it calculates and then combines with weights, selecting the best models, and it uses a hash table when calculating probabilities to make sure that the computer can keep track of everything. By artfully combining some of the best lossless compression algorithms with unique software engineering techniques, Crinkler has become one of the most popular demotools in the Demoscene, an invaluable tool used by many to continue to push their technology and skills past their limits.

References

- Christensen, Aske. 2005. “Crinkler - compressing Windows 4k intros to EXE files,” July. <https://www.youtube.com/watch?v=VIzg9NCW3rs>.
- Mahoney, Matthew V. 2002. “The PAQ1 Data Compression Program,” March 19, 2002. <http://mattmahoney.net/dc/paq1.pdf>.
- Miller, Jeffrey W. 2011. “(IC 5.1) Arithmetic coding - introduction,” September 24, 2011. <https://www.youtube.com/watch?v=ouYV3rBtrTI>.
- Mutel, Alexandre. 2010. “Crinkler secrets, 4k intro executable compressor at its best,” December 28, 2010. <http://xoofx.com/blog/2010/12/28/crinkler-secrets-4k-intro-executable/>.
- Sonntag, Lawrence, and Patrick Brown. 2019. “Demoscene: The Strangest (and Coolest) Computer Subculture - Inside Gaming Explains,” December 8, 2019. <https://www.youtube.com/watch?v=VENFVhQqGls>.